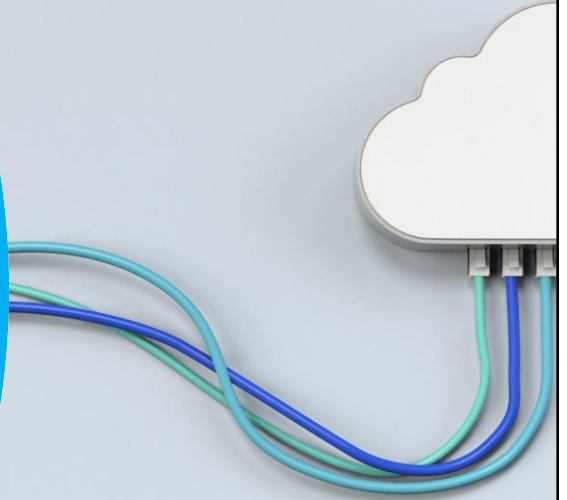


Transportschicht

NETZWERKTECHNIK / SEMESTER 1 UND 2



1

AGENDA

- 01 TRANSPORT VON DATEN
- 02 TCP ÜBERBLICK
- 03 UDP ÜBERBLICK
- 04 PORTNUMMERN
- 05 VERBINDUNGSaufbau UND ABBAU
- 06 ZUVERLÄSSIGKEIT UND FLUßKONTROLLE
- 07 UDP KOMMUNIKATION
- 08 APPENDIX

2

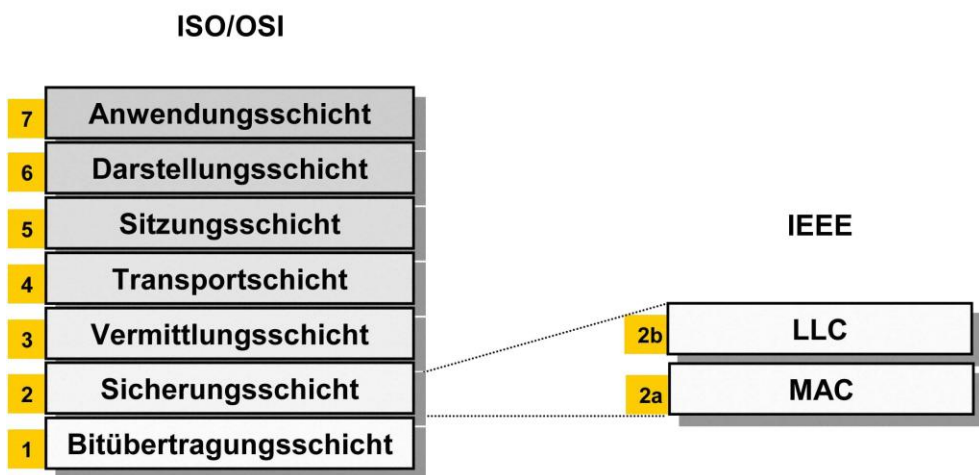
01

Transport von Daten

3

NETZWERKTECHNIK / SEMESTER 1 und 2

Einordnung im ISO/OSI Modell

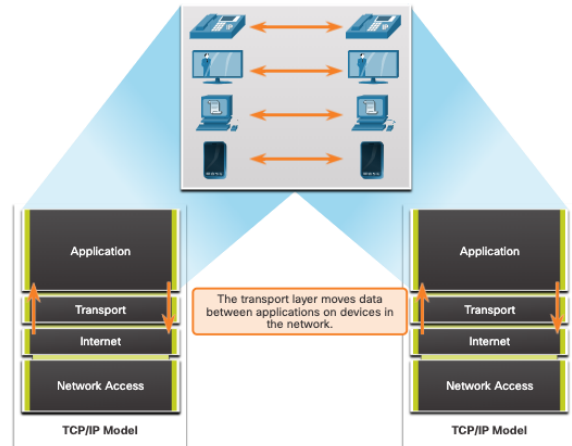


4

Rolle der Transportschicht

Die Transportschicht ist:

- verantwortlich für die **logische Kommunikation** zwischen Anwendungen, die auf verschiedenen Hosts ausgeführt werden.
- die **Verbindung zwischen der Anwendungsschicht und den unteren Schichten**, die für die Netzwerkübertragung verantwortlich sind.

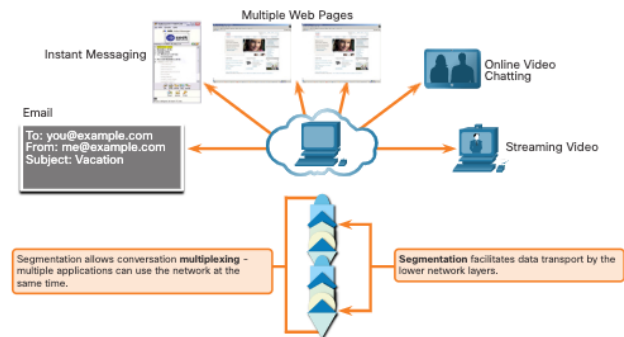


5

Verantwortung der Transportschicht

Die Transportschicht hat folgende Aufgaben:

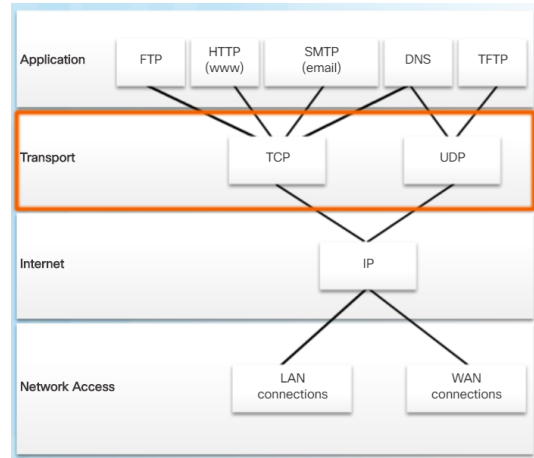
- Verfolgen einzelner **Konversationen**
- Segmentieren** von Daten und erneutes Zusammenfügen von Segmenten
- hinzufügen von **Kopfzeileninformationen**
- Identifizieren, Trennen und Verwalten mehrerer Konversationen
- Verwendet **Segmentierung und Multiplexing**, um die Verschachtelung verschiedener Kommunikationsgespräche im selben Netzwerk zu ermöglichen



6

Protokolle der Transportschicht

- Internet Protokoll (IP) spezifiziert nicht, wie die Zustellung oder der Transport der Pakete erfolgt.
- Protokolle der Transportschicht geben an, wie Nachrichten zwischen Hosts übertragen werden, und sind für die **Verwaltung der Zuverlässigkeitsanforderungen einer Konversation verantwortlich**.
- Die Transportschicht umfasst die Protokolle **TCP** und **UDP**.

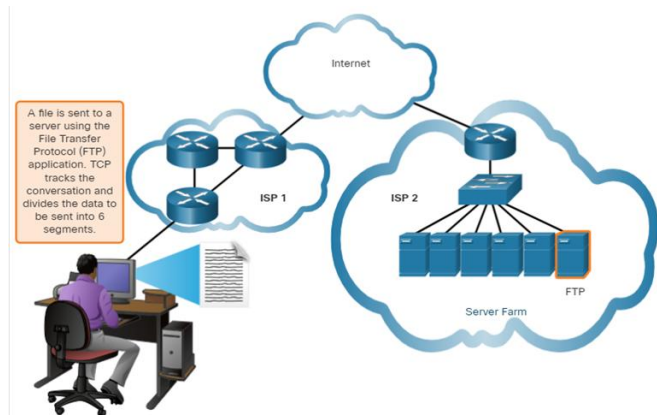


Transmission Control Protocol (TCP)

- TCP bietet **Zuverlässigkeit und Flusskontrolle**.

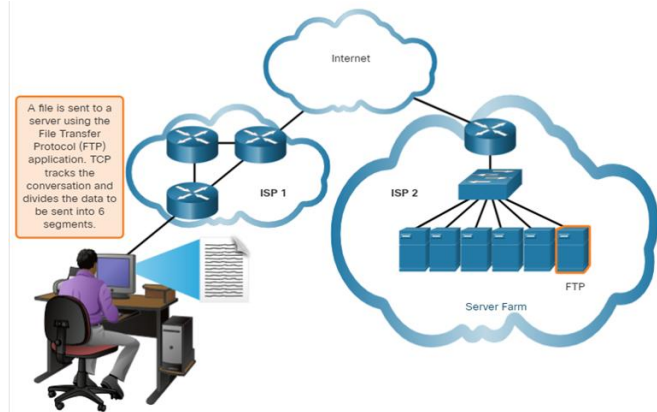
Grundlegende TCP-Operationen:

- Numerisieren und verfolgen von Datensegmenten**, die von einer bestimmten Anwendung an einen bestimmten Host übertragen werden
- Empfangene Daten bestätigen**
- Erneute Übermitteln von nicht bestätigten Daten** nach einer bestimmten Zeit
- Daten, die möglicherweise in der falschen Reihenfolge eintreffen, **sequenzieren**.
- Daten mit einer effizienten, für den Empfänger **akzeptablen Rate** senden.



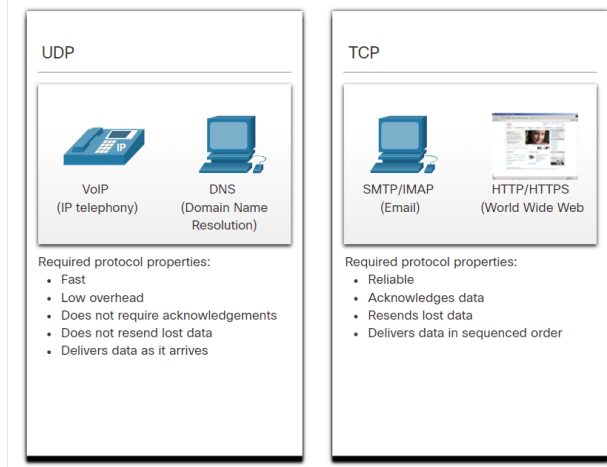
User Datagram Protocol (UDP)

- UDP bietet die grundlegenden Funktionen für die Bereitstellung von Datagrammen zwischen den entsprechenden Anwendungen, mit **sehr geringem Overhead und Datenprüfung**.
- UDP ist ein **verbindungsloses Protokoll**.
- UDP ist als **Best-Effort-Übermittlungsprotokoll** bekannt, da es keine Bestätigung gibt, dass die Daten am Ziel empfangen werden.



Das richtige Transport-Layer-Protokoll für die richtige Anwendung

- UDP wird auch von Request-and-Reply-Anwendungen verwendet, bei denen der Datenverbrauch minimal ist und die erneute Übertragung schnell erfolgen kann.
- Wenn es wichtig ist, dass alle Daten ankommen und in der richtigen Reihenfolge verarbeitet werden können, wird TCP als Transportprotokoll verwendet.



02

TCP-Überblick

11

NETZWERKTECHNIK / SEMESTER 1 und 2

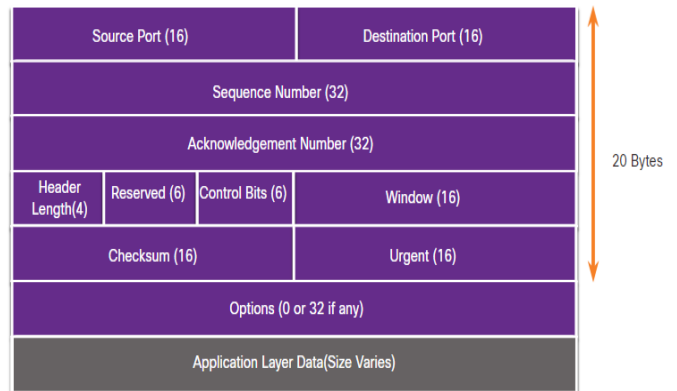
TCP Features

- Richtet eine Sitzung ein - TCP ist ein **verbindungsorientiertes Protokoll**, das eine permanente Verbindung (oder Sitzung) zwischen Quell- und Zielgeräten aushandelt und aufbaut, bevor Datenverkehr weitergeleitet wird.
- Gewährleistet eine zuverlässige Lieferung - Aus vielen Gründen ist es möglich, dass ein Segment beschädigt wird oder vollständig verloren geht, wenn es über das Netzwerk übertragen wird. TCP **stellt sicher, dass jedes Segment, das von der Quelle gesendet wird, am Ziel ankommt**.
- Bietet Lieferung in derselben Reihenfolge - Da Netzwerke möglicherweise mehrere Routen mit unterschiedlichen Übertragungsraten bereitstellen, können **Daten in der falschen Reihenfolge ankommen**.
- Unterstützt Flusskontrolle - Netzwerk-Hosts verfügen über begrenzte Ressourcen (d. h. Arbeitsspeicher und Rechenleistung). Wenn TCP erkennt, dass diese Ressourcen überlastet sind, kann es anfordern, dass die **sendende Anwendung die Geschwindigkeit des Datenflusses reduziert**.

12

TCP Header

- TCP ist ein zustandsbehaftetes Protokoll, was bedeutet, dass es den Status der Kommunikationssitzung verfolgt.
- TCP zeichnet auf, welche Informationen es gesendet hat und welche Informationen bestätigt wurden.

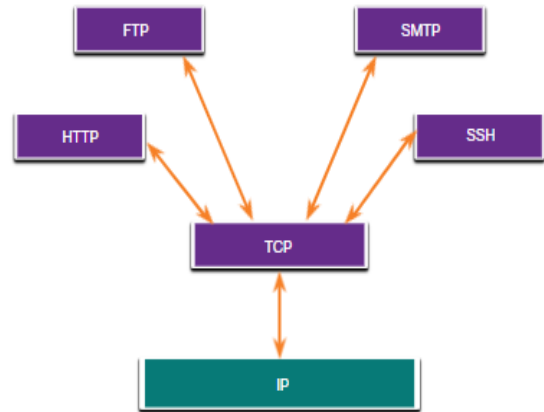


Felder des TCP Header

TCP Header Field	Description
Source Port	16-Bit-Feld, das zur Identifizierung der Quellenanwendung anhand der Portnummer verwendet wird.
Destination Port	16-Bit-Feld, das zur Identifizierung der Zielanwendung anhand der Portnummer verwendet wird.
Sequence Number	32-Bit-Feld, das für die Datenreassemblierung verwendet wird.
Acknowledgment Number	32-Bit-Feld, das verwendet wird, um anzuzeigen, dass Daten empfangen wurden und das nächste Byte von der Quelle erwartet wird.
Header Length	4-Bit-Feld, das als "Datenoffset" bezeichnet wird und die Länge des TCP-Segmentheaders angibt.
Reserved	6-Bit-Feld, das für die zukünftige Verwendung reserviert ist.
Control bits	Verwendetes 6-Bit-Feld, das Bitcodes oder Flags enthält, die den Zweck und die Funktion des TCP-Segments angeben.
Window size	16-Bit-Feld, das verwendet wird, um die Anzahl der Bytes anzugeben, die gleichzeitig akzeptiert werden können.
Checksum	16-Bit-Feld, das für die Fehlerprüfung des Segmentkopfes und der Daten verwendet wird.
Urgent	16-Bit-Feld, das verwendet wird, um anzuzeigen, ob die enthaltenen Daten dringend sind.

Anwendungen die TCP nutzen

TCP übernimmt alle Aufgaben, die mit der Unterteilung des Datenstroms in Segmente, der Bereitstellung von Zuverlässigkeit, der Steuerung des Datenflusses und der Neuordnung von Segmenten verbunden sind.



03

UDP-Überblick

UDP Features

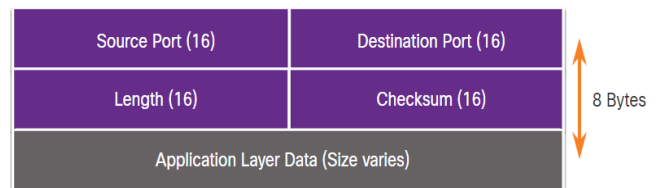
Zu den UDP-Funktionen gehören die folgenden:

- Die Daten werden in der **Reihenfolge ihres Eingangs rekonstruiert**.
- Segmente, die verloren gehen, werden **nicht erneut gesendet**.
- Es gibt **keine Sitzungseinrichtung**.
- Der Sender wird **nicht über die Verfügbarkeit der Ressource** informiert.

17

UDP Header

Der UDP-Header ist **viel einfacher als der TCP-Header**, da er nur vier Felder hat und 8 Byte (d.h. 64 Bit) benötigt.



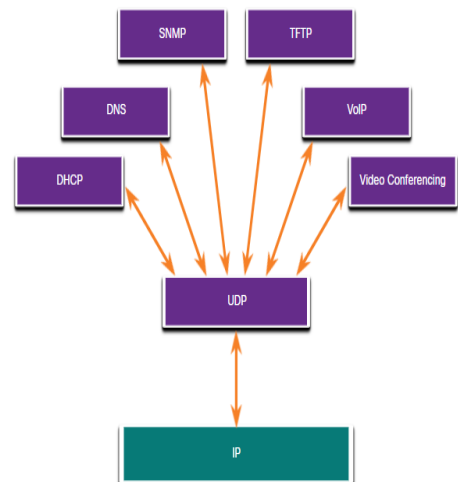
18

Felder des UDP Header

UDP Header Field	Description
Source Port	6-Bit-Feld, das zur Identifizierung der Quellanwendung anhand der Portnummer verwendet wird.
Destination Port	16-Bit-Feld, das zur Identifizierung der Zieranwendung anhand der Portnummer verwendet wird.
Length	16-Bit-Feld, das die Länge des UDP-Datagramm-Headers angibt.
Checksum	16-Bit-Feld, das für die Fehlerüberprüfung des Datagramm-Headers und der Daten verwendet wird.

Anwendungen die UDP nutzen

- **Live-Video- und Multimedia-Anwendungen** - Diese Anwendungen können einen gewissen Datenverlust tolerieren, erfordern jedoch nur eine geringe oder keine Verzögerung. Beispiele hierfür sind VoIP und Live-Streaming-Videos.
- **Einfache Anforderungs- und Antwortanwendungen:** Anwendungen mit einfachen Transaktionen, bei denen ein Host eine Anforderung sendet und möglicherweise keine Antwort erhält. Beispiele hierfür sind DNS und DHCP.
- **Anwendungen, die die Zuverlässigkeit selbst handhaben** - Unidirektionale Kommunikation, bei der Flusssteuerung, Fehlererkennung, Bestätigungen und Fehlerbehebung nicht erforderlich sind oder von der Anwendung verarbeitet werden können. Beispiele hierfür sind SNMP und TFTP.



04

Portnummern

21

NETZWERKTECHNIK / SEMESTER 1 und 2

Mehrere separate Kommunikationen

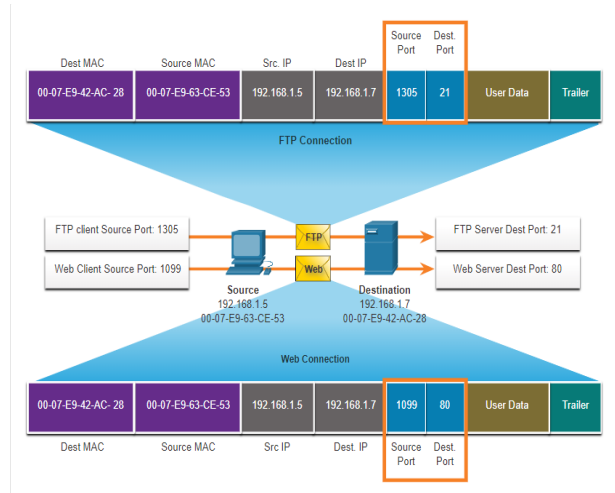
- TCP- und UDP-Transportschichtprotokolle verwenden **Portnummern, um mehrere, gleichzeitige Konversationen zu verwalten.**
- Die **Quellportnummer** ist mit der ursprünglichen Anwendung auf dem lokalen Host verknüpft, während die **Zielportnummer** mit der Zielanwendung auf dem Remotehost verknüpft ist.



22

Socket Paare

- Die Quell- und Zielports werden innerhalb des Segments platziert.
- Die Segmente werden dann in einem IP-Paket gekapselt.
- Die Kombination aus der **Quell-IP-Adresse und der Quellportnummer oder der Ziel-IP-Adresse und der Zielportnummer** wird als **Socket** bezeichnet.
- Sockets ermöglichen es **mehreren Prozessen, die auf einem Client ausgeführt werden, sich voneinander zu unterscheiden**, und mehreren Verbindungen zu einem Serverprozess, um voneinander zu unterscheiden.



Gruppen von Portnummern

Port Group	Number Range	Description
Well-known Ports	0 to 1,023	- Diese Portnummern sind für gängige oder beliebte Dienste und Anwendungen wie Webbrowser, E-Mail-Clients und RAS-Clients reserviert. - Definierte bekannte Ports für gängige Serveranwendungen ermöglichen es Clients, den zugehörigen Dienst leicht zu identifizieren.
Registered Ports	1,024 to 49,151	- Diese Portnummern werden von der IANA einer anfordernden Entität zur Verwendung mit bestimmten Prozessen oder Anwendungen zugewiesen. - Bei diesen Prozessen handelt es sich in erster Linie um einzelne Anwendungen, die ein Benutzer installiert hat, und nicht um allgemeine Anwendungen, die eine bekannte Portnummer erhalten. - Cisco hat beispielsweise den Port 1812 für den Authentifizierungsprozess des RADIUS-Servers registriert.
Private and/or Dynamic Ports	49,152 to 65,535	- Diese Ports werden auch als ephemere Ports bezeichnet. - Das Betriebssystem des Clients weist in der Regel dynamisch Portnummern zu, wenn eine Verbindung zu einem Dienst initiiert wird. - Der dynamische Port wird dann verwendet, um die Clientanwendung während der Kommunikation zu identifizieren.

„Well-Known“ Portnummern

Port Number	Protocol	Application
20	TCP	File Transfer Protocol (FTP) - Data
21	TCP	File Transfer Protocol (FTP) - Control
22	TCP	Secure Shell (SSH)
23	TCP	Telnet
25	TCP	Simple Mail Transfer Protocol (SMTP)
53	UDP, TCP	Domain Name Service (DNS)
67	UDP	Dynamic Host Configuration Protocol (DHCP) - Server
68	UDP	Dynamic Host Configuration Protocol - Client
69	UDP	Trivial File Transfer Protocol (TFTP)
80	TCP	Hypertext Transfer Protocol (HTTP)
110	TCP	Post Office Protocol version 3 (POP3)
143	TCP	Internet Message Access Protocol (IMAP)
161	UDP	Simple Network Management Protocol (SNMP)
443	TCP	Hypertext Transfer Protocol Secure (HTTPS)

netstat Kommando

Unerklärliche TCP-Verbindungen können eine große Sicherheitsbedrohung darstellen. **netstat** ist ein wichtiges Werkzeug zur Überprüfung von Verbindungen.

```
C:\> netstat
Active Connections
Proto Local Address           Foreign Address         State
TCP    192.168.1.124:3126      192.168.0.2:netbios-ssn ESTABLISHED
TCP    192.168.1.124:3158      207.138.126.152:http   ESTABLISHED
TCP    192.168.1.124:3159      207.138.126.169:http   ESTABLISHED
TCP    192.168.1.124:3160      207.138.126.169:http   ESTABLISHED
TCP    192.168.1.124:3161      sc.msn.com:http        ESTABLISHED
TCP    192.168.1.124:3166      www.cisco.com:http      ESTABLISHED
```

05

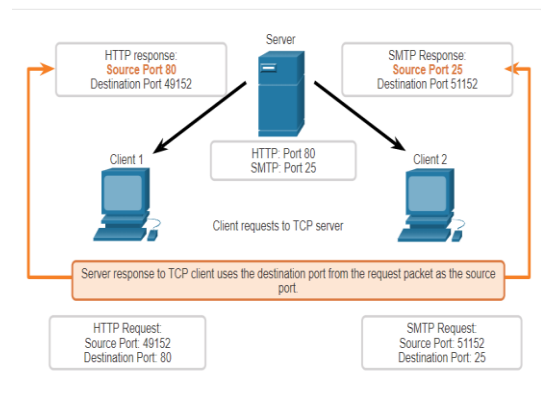
Verbindungsaufbau und Abbau

27

NETZWERKTECHNIK / SEMESTER 1 und 2

TCP Server Prozess

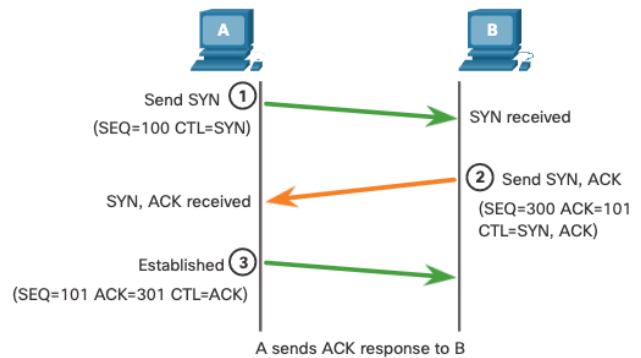
- **Jeder Anwendungsprozess, der auf einem Server ausgeführt wird, ist so konfiguriert, dass er eine Portnummer verwendet.**
- Einem einzelnen Server können nicht zwei Dienste zugewiesen werden, die derselben Portnummer innerhalb derselben Transportschichtdienste zugewiesen sind.
- Eine aktive Serveranwendung, die einem bestimmten Port zugewiesen ist, gilt als offen, was bedeutet, dass die Transportschicht an diesen Port adressierte Segmente akzeptiert und verarbeitet.
- Jede eingehende Clientanforderung, die an den richtigen Socket adressiert ist, wird akzeptiert, und die Daten werden an die Serveranwendung übergeben.



28

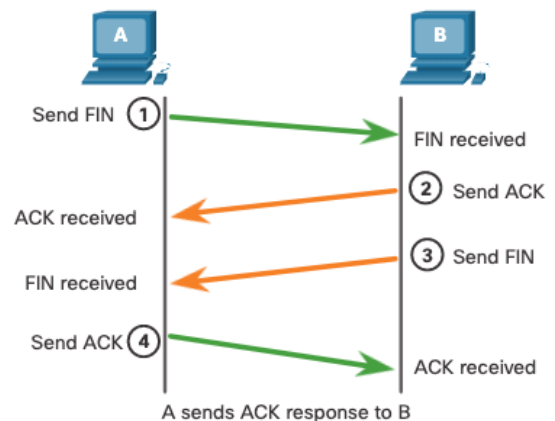
TCP Verbindungsaufbau

- **Schritt 1:** Der initiiierende Client fordert eine Client-zu-Server-Kommunikationssitzung mit dem Server an.
- **Schritt 2:** Der Server bestätigt die Client-zu-Server-Kommunikationssitzung und fordert eine Server-zu-Client-Kommunikationssitzung an.
- **Schritt 3:** Der initiiierende Client bestätigt die Server-zu-Client-Kommunikationssitzung.



Session Termination

- **Schritt 1:** Wenn der Client keine Daten mehr im Stream senden kann, sendet er ein Segment mit dem FIN-Flag.
- **Schritt 2:** Der Server sendet eine ACK, um den Empfang der FIN zu bestätigen und die Sitzung vom Client zum Server zu beenden.
- **Schritt 3:** Der Server sendet eine FIN an den Client, um die Server-zu-Client-Sitzung zu beenden.
- **Schritt 4:** Der Client antwortet mit einer ACK, um die FIN vom Server zu bestätigen.

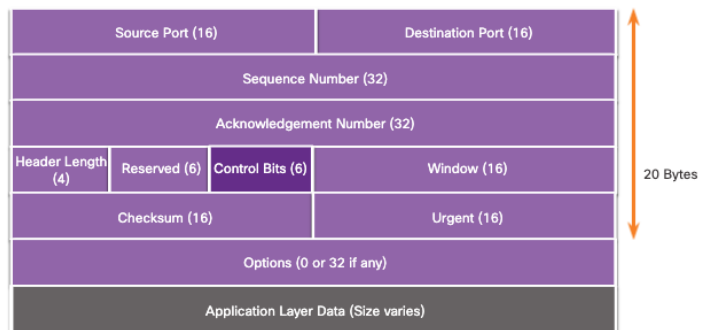


TCP 3-Way Handshake

- Es stellt fest, dass das **Zielgerät im Netzwerk vorhanden** ist.
- Er überprüft, ob das **Zielgerät über einen aktiven Dienst verfügt** und Anforderungen an die Zielporthnummer akzeptiert, die der initiiierende Client verwenden möchte.
- Er informiert das Zielgerät, dass der Quellclient beabsichtigt, eine **Kommunikationssitzung über diese Portnummer** einzurichten.
- Nach Abschluss der Kommunikation werden die Sitzungen geschlossen und die Verbindung beendet. Die Verbindungs- und Sitzungsmechanismen ermöglichen die TCP-Zuverlässigkeitsfunktion.

TCP 3-Way Handshake Analyse

- URG** - Urgent pointer field significant
- ACK** - Acknowledgment flag used in connection establishment and session termination
- PSH** - Push function
- RST** - Reset the connection when an error or timeout occurs
- SYN** - Synchronize sequence numbers used in connection establishment
- FIN** - No more data from sender and used in session termination



Video - TCP 3-Way Handshake Analyse

No.	Time	Source	Destination	Protocol	Info
10	16.303490	10.1.1.1	192.168.254.254	TCP	kiosk > http [SYN] Seq=0 W
11	16.304896	192.168.254.254	10.1.1.1	TCP	http > kiosk [SYN, ACK] Seq=1 A
12	16.304925	10.1.1.1	192.168.254.254	TCP	kiosk > http [ACK] Seq=1 A
13	16.305132	10.1.1.1	192.168.254.254	HTTP	GET / HTTP/1.1
14	16.307875	192.168.254.254	10.1.1.1	TCP	http > kiosk [ACK] Seq=1 A

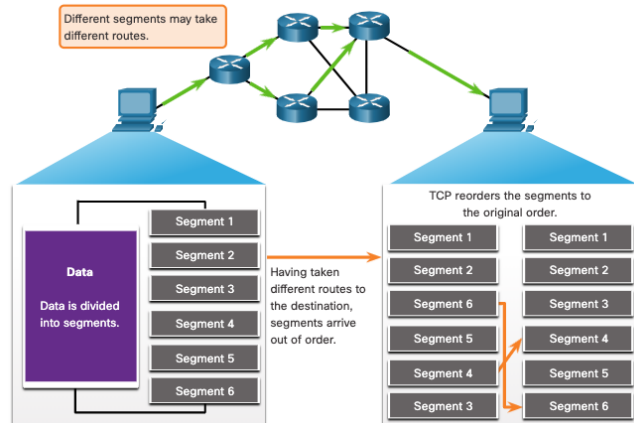
Frame 10: 62 bytes on wire (496 bits), 62 bytes captured (496 bits)	
Ethernet II, Src: VMware_bei02:88 (00:50:56:b6:02:88), Dst: Cisco_63:74:a0 (00:0f:12:41:63:74)	
Internet Protocol Version 4, Src: 10.1.1.1 (10.1.1.1), Dst: 192.168.254.254 (192.168.254.254)	
Transmission Control Protocol, Src Port: kiosk (1061), Dst Port: http (80), Seq: 0, Len: 0	
Source port: kiosk (1061) Destination port: http (80) [Stream index: 0] Sequence number: 0 (relative sequence number) Header length: 28 bytes	
Flags: 0x02 (SYN)	
000. = Reserved: Not set ...0. = Nonce: Not set 0... = Congestion window reduced (CWR): Not set 0... = ECN-Echo: Not set 00.. = Urgent: Not set 0... = Acknowledgement: Not set 0... = Push: Not set 0... = Reset: Not set1.. = Syn: Set 0... = Fin: Not set	

06

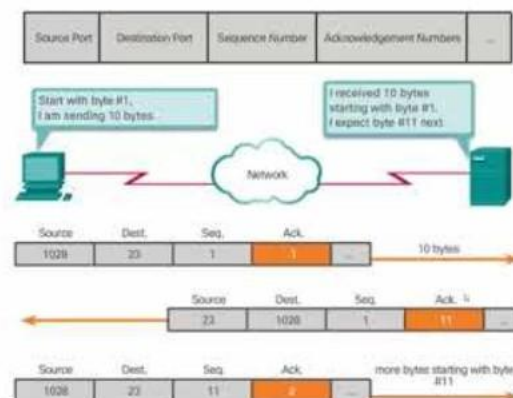
Zuverlässigkeit und Flusskontrolle

TCP-Zuverlässigkeit - Garantierte und geordnete Übertragung

- TCP kann auch dazu beitragen, den Paketfluss aufrechtzuerhalten, damit die Geräte nicht überlastet werden.
- Es kann vorkommen, dass TCP-Segmente nicht an ihrem Ziel ankommen oder nicht in der richtigen Reihenfolge ankommen.
- Alle Daten müssen empfangen werden, und die Daten in diesen Segmenten müssen wieder in die ursprüngliche Reihenfolge eingefügt werden.**
- Um dieses Ziel zu erreichen, werden im Header jedes Pakets **Sequenznummern** zugewiesen.

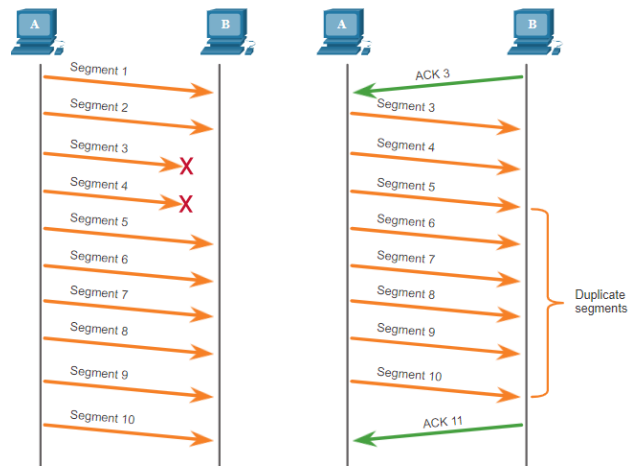


Video - TCP Reliability - Sequence Numbers und Acknowledgments



TCP Reliability – Data Loss und Retransmission

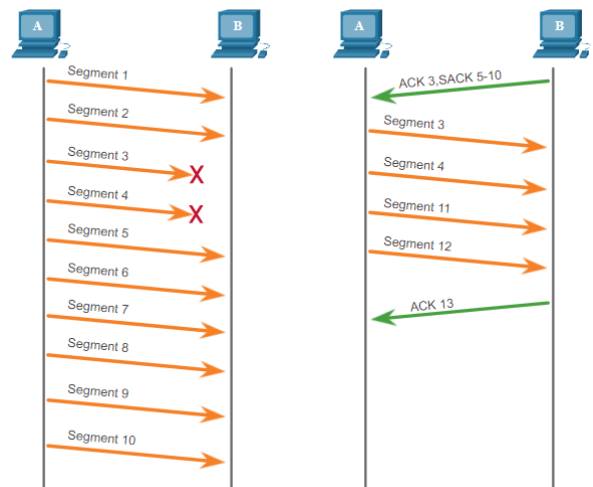
- Gelegentlich kommt es zu **Datenverlusten**.
- TCP bietet Methoden zur Verwaltung dieser Segmentverluste.
- Dazu gehört ein Mechanismus zur **erneuten Übertragung von Segmenten für nicht bestätigte Daten**.



37

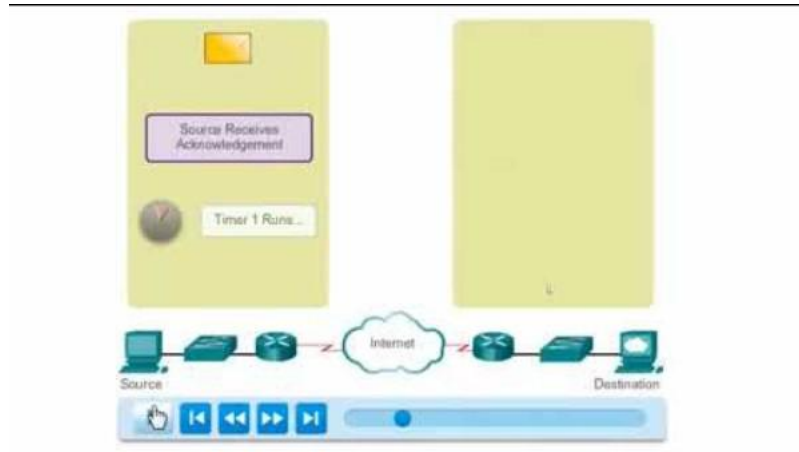
TCP Reliability – Data Loss and Retransmission

- Hostbetriebssysteme verwenden heute in der Regel eine optionale TCP-Funktion namens **Selective Conknowledgment (SACK)**, die während des Drei-Wege-Handshakes ausgehandelt wird.
- Wenn beide Hosts SACK unterstützen, kann der Empfänger explizit bestätigen, welche Segmente (Bytes) empfangen wurden, einschließlich etwaiger diskontinuierlicher Segmente.



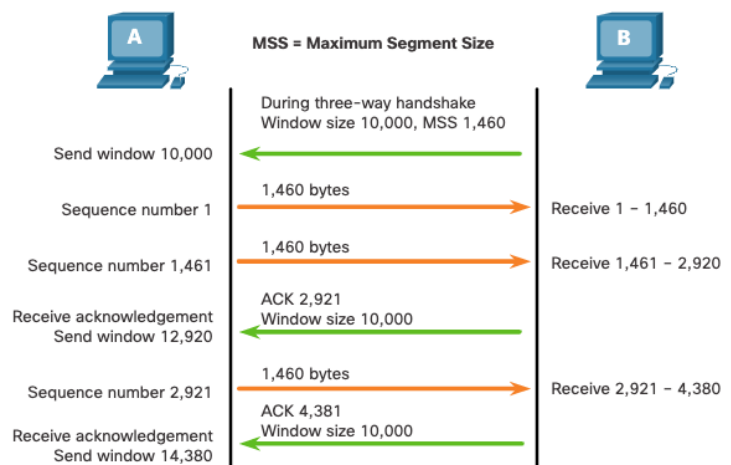
38

Video - TCP Reliability – Data Loss und Retransmission



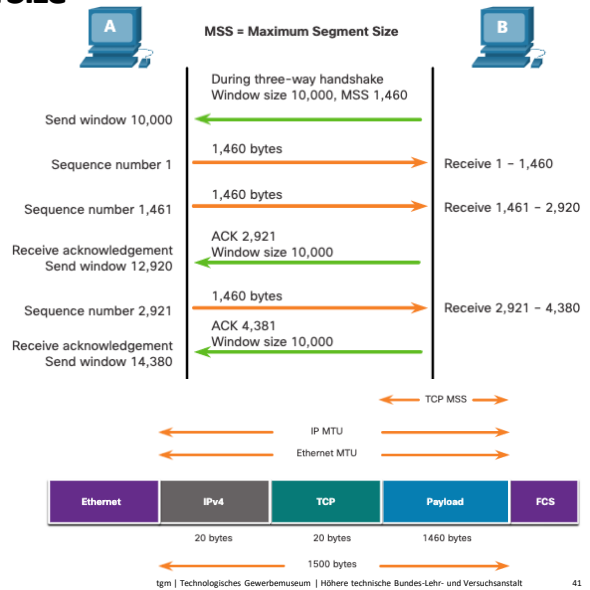
TCP Flow Control – Window Size und Acknowledgments

- Die Flusststeuerung ist die Datenmenge, die das Ziel zuverlässig empfangen und verarbeiten kann.
- Die Flusststeuerung trägt dazu bei, die Zuverlässigkeit der TCP-Übertragung aufrechtzuerhalten, indem die **Geschwindigkeit des Datenflusses zwischen Quelle und Ziel für eine bestimmte Sitzung angepasst** wird.



TCP Flow Control – Maximum Segment Size

- Die maximale Segmentgröße (MSS) ist die **maximale Datenmenge, die das Zielgerät empfangen kann**.
- Eine **gängige MSS-Datei ist 1.460 Byte** bei Verwendung von IPv4.
- Ein Host bestimmt den Wert seines MSS-Feldes, indem er die IP- und TCP-Header von der Ethernet Maximum Transmission Unit (MTU) subtrahiert, die standardmäßig 1500 Byte beträgt.
- 1500 minus 40 (20 Byte für den IPv4-Header und 20 Byte für den TCP-Header) lässt 1460 Byte übrig.

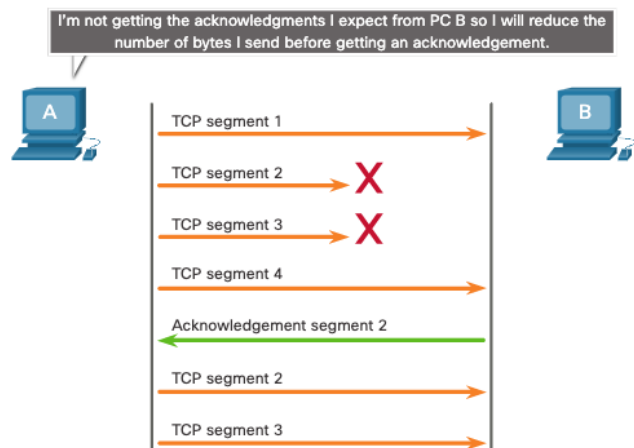


tgm [Quelle: Introduction to Networks v7.0 (ITN), Cisco Systems]

41

TCP Flow Control – Congestion Avoidance

- Wenn in einem Netzwerk eine Überlastung auftritt, führt dies dazu, dass Pakete vom überlasteten Router verworfen werden.
- Um Überlastungen zu vermeiden und zu kontrollieren, verwendet TCP mehrere Überlastungsbehandlungsmechanismen, Timer und Algorithmen.



tgm [Quelle: Introduction to Networks v7.0 (ITN), Cisco Systems]

tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

42

42

07

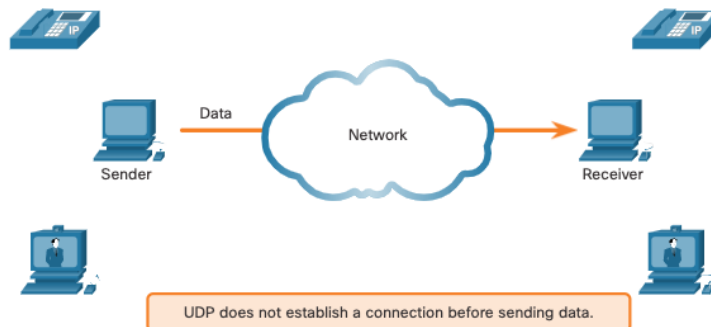
UDP Kommunikation

43

NETZWERKTECHNIK / SEMESTER 1 und 2

Geringer UDP-Overhead im Vergleich zu Zuverlässigkeit

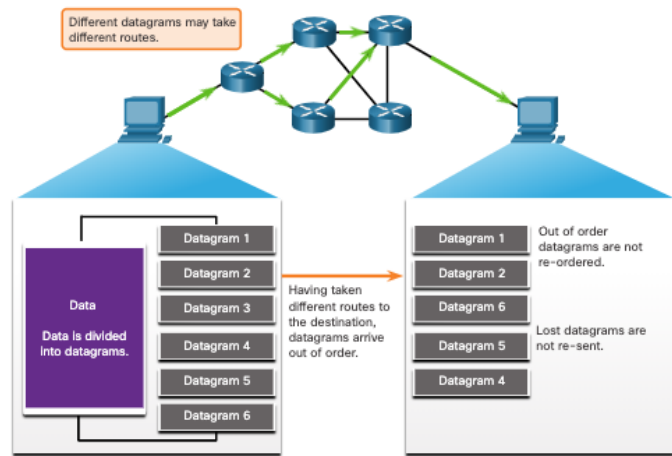
UDP stellt keine Verbindung her. UDP bietet einen geringen Overhead für den Datentransport, da es über einen kleinen Datagramm-Header und keinen zusätzlichen Overhead für die Netzwerkverwaltung verfügt.



44

Zusammenbau von UDP-Datagrammen

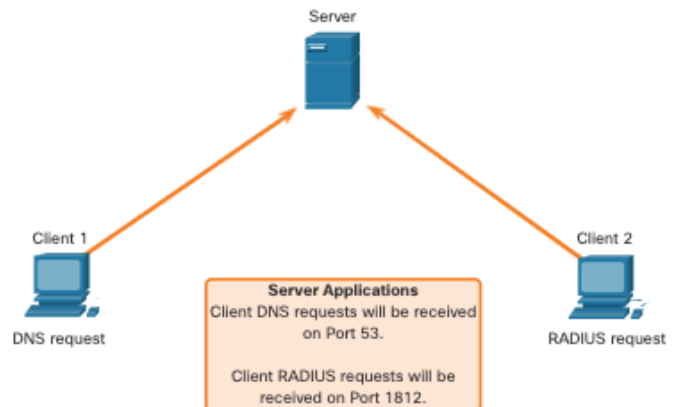
- UDP verfolgt Sequenznummern nicht wie TCP.
- UDP hat keine Möglichkeit, die Datagramme in ihrer Übertragungsreihenfolge neu anzuordnen.
- UDP setzt die Daten einfach in der Reihenfolge des Empfangs wieder zusammen und leitet sie an die Anwendung weiter.



45

UDP-Serverprozesse und Anforderungen

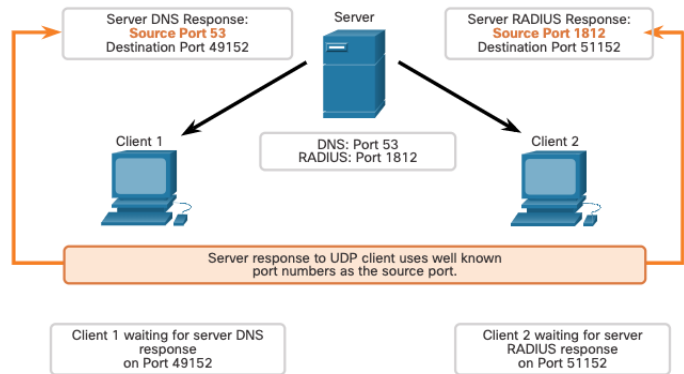
- UDP-basierten Serveranwendungen werden bekannte oder **registrierte Portnummern** zugewiesen.
- UDP empfängt ein Datagramm, das für einen dieser Ports bestimmt ist, und leitet die Anwendungsdaten anhand seiner Portnummer an die entsprechende Anwendung weiter.



46

UDP-Client Prozesse

- Der UDP-Clientprozess wählt dynamisch eine Portnummer aus dem Bereich der Portnummern aus und verwendet diese als Quellport für die Konversation.
- Der Zielport ist in der Regel die bekannte oder registrierte Portnummer, die dem Serverprozess zugewiesen ist.
- Nachdem ein Client die Quell- und Zielports ausgewählt hat, wird dasselbe Portpaar im Header aller Datagramme in der Transaktion verwendet.



08

Deep-Dive

Aufgaben der Transportschicht

49

NETZWERKTECHNIK / SEMESTER 1 und 2

Aufgaben der Transportschicht

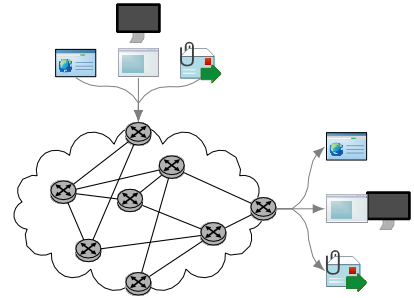
Die wesentlichen Aufgaben der Transportschicht sind

- **Multiplexing** von Datenströmen unterschiedlicher Anwendungen bzw. Anwendungsinstanzen,
- Bereitstellung **verbindungsloser** und **verbindungsorientierter** Transportmechanismen und
- Mechanismen zur **Stau-** und **Flusskontrolle**.

50

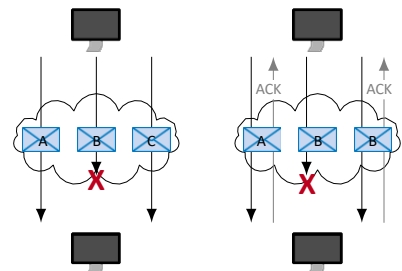
Aufgaben der Transportschicht - Multiplexing

- Segmentierung der Datenströme unterschiedlicher Anwendungen (Browser, Chat, Email, ...)
- Segmente werden in jeweils unabhängigen IP-Paketen zum Empfänger geroutet
- Empfänger muss die Segmente den einzelnen Datenströmen zuordnen und an die jeweilige Anwendung weiterreichen



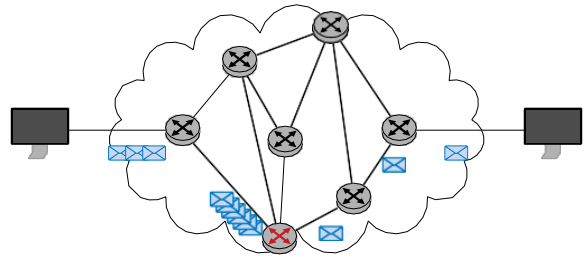
Aufgaben der Transportschicht - Transportdienste

- Verbindungslos (Best Effort)**
 - Segmente sind aus Sicht der Transportschicht voneinander unabhängig
 - Keine Sequenznummern, keine Übertragungswiederholung, keine Garantie der richtigen Reihenfolge
- Verbindungsorientiert**
 - Übertragungswiederholung bei Fehlern
 - Garantie der richtigen Reihenfolge einzelner Segmente



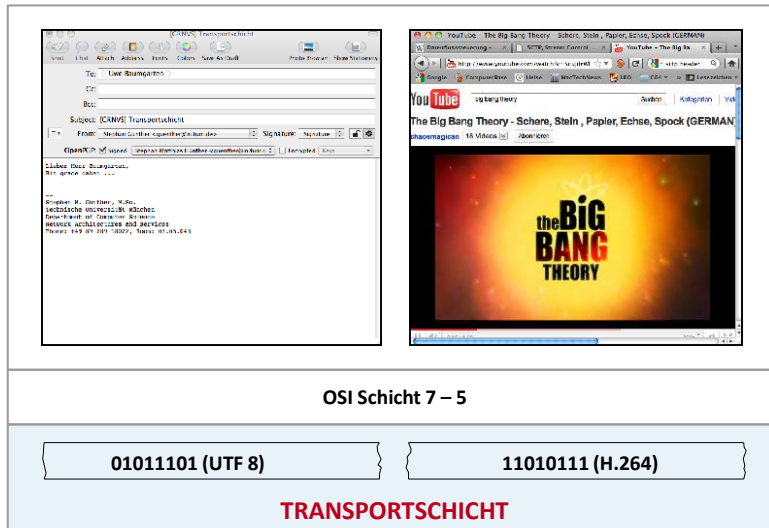
Aufgaben der Transportschicht - Stau- und Flusskontrolle

- **Staukontrolle (Congestion Control)**
 - Reaktion auf drohende Überlast im Netz
- **Flusskontrolle (Flow Control)**
 - Laststeuerung durch den Empfänger



Multiplexing

Multiplexing



tgm [Quelle: Grundlagen Rechnernetze und Verteilte Systeme (GRNVS), Prof. Dr.-Ing. Georg Carle, TUM, 2025]

tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

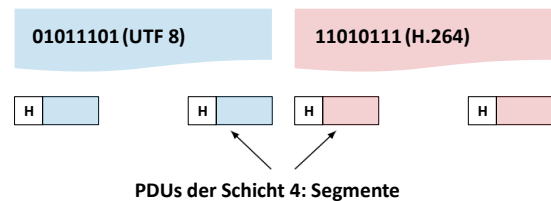
55

55

Multiplexing

Auf der Transportschicht

- werden die kodierten Datenströme in **Segmente** unterteilt und
- jedes Segment mit einem **Header** versehen.



Ein solcher Header enthält jeweils mindestens

- einen **Quellport** und
- einen **Zielport**,

welche zusammen mit den Absender- bzw. Empfänger-IP und dem verwendeten Transportprotokoll die Anwendung auf am Sender bzw. Empfänger jeweils eindeutig identifizieren.

⇒ 5-Tupel bestehend aus:

(SrcIPAddr, SrcPort, DstIPAddr, DstPort, Protocol)

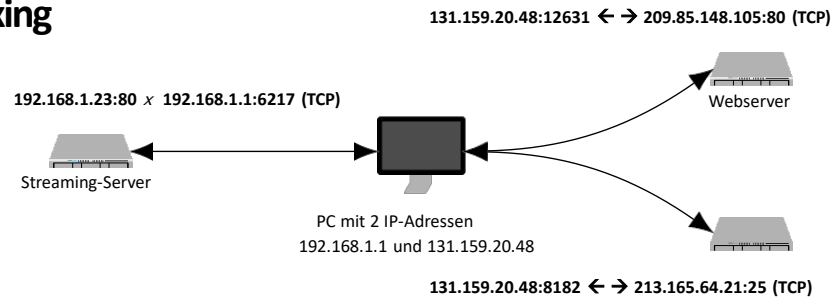
tgm [Quelle: Grundlagen Rechnernetze und Verteilte Systeme (GRNVS), Prof. Dr.-Ing. Georg Carle, TUM, 2025]

tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

56

56

Beispiel für Multiplexing



- Portnummern sind bei den bekannten Transportprotokollen 16 bit lang.
- Betriebssysteme verwenden das 5-Tupel (IP-Adressen, Portnummern, Protokoll), um Anwendungen **Sockets** bereitzustellen.
- Eine Anwendung wiederum adressiert einen Socket mittels eines **File-Deskriptors** (ganzzahliger Wert).
- Verbindungsorientierte Sockets können nach dem Verbindungsaufbau sehr einfach genutzt werden, da der Empfänger bereits feststeht (Lesen und Schreiben mittels Systemaufrufen `read()` und `write()` möglich).
- Verbindungslose Sockets benötigen Adressangaben, an wen gesendet oder von wem empfangen werden soll (`sendto()` und `recvfrom()`).

Verbindungslose Übertragung

Verbindungslose Übertragung

Funktionsweise: Header eines Transportprotokolls besteht mind. aus

- **Quell- und Zielport sowie**
- **einer Längenangabe der Nutzdaten.**

Dies ermöglicht es einer Anwendung beim Senden für jedes einzelne Paket

- den Empfänger (IP-Adresse) und
- die empfangende Anwendung (Protokoll und Zielport) anzugeben.

Probleme: Da die Segmente unabhängig voneinander und aus Sicht der Transportschicht **zustandslos** versendet werden, kann nicht sichergestellt werden, dass

- Segmente den Empfänger erreichen (Pakete können verloren gehen) und
- der Empfänger die Segmente in der richtigen Reihenfolge erhält (Pakete werden unabhängig geroutet).

Folglich spricht man von einer **ungesicherten, verbindungslosen** oder **nachrichtenorientierten** Kommunikation. (Nicht zu verwechseln mit nachrichtenorientierter Übertragung auf Schicht 2)

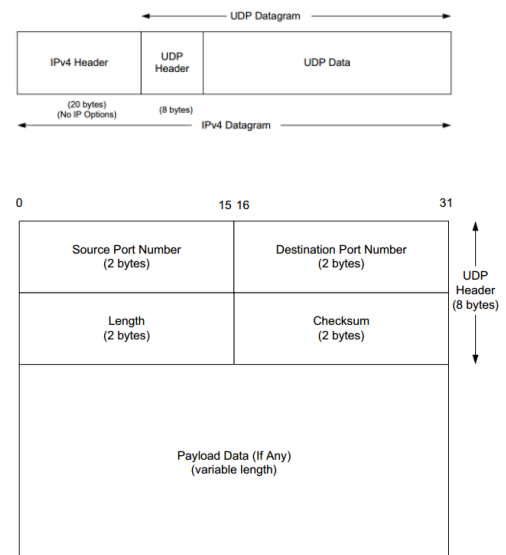
User Datagram Protocol (UDP)

User Datagram Protocol (UDP) ist eines der beiden am häufigsten verwendeten Transportprotokolle im Internet. Es bietet

- ungesicherte und nachrichtenorientierte Übertragung
- bei geringem Overhead.

Header:

- „**Length**“ gibt die Länge von Header und Daten in Vielfachen von Byte an.
- Die **Prüfsumme** erstreckt sich über Header und Daten.
 - Die Verwendung der UDP-Prüfsumme ist bei IPv4 optional, wird für IPv6 jedoch vorausgesetzt.
 - Wird sie nicht verwendet, wird das Feld auf 0 gesetzt.
 - Wird sie verwendet, wird zur Berechnung ein Pseudo-Header genutzt (eine Art „Default-IP-Header“ der nur zur Berechnung der Prüfsumme dient). Er beinhaltet folgende Felder des IP-Headers: Quell- und Ziel-IP-Adresse, ein 8 bit langes Feld mit Nullen, Protocol-ID und Länge des UDP- Datagramms.



Vorteil und Nachteile von UDP

Vorteile von UDP:

- Geringer Overhead
- Keine Verzögerung durch Verbindungsaufbau oder Retransmits und Reordering von Segmenten
- Gut geeignet für Echtzeitanwendungen (Voice over IP, Online-Spiele) sofern gelegentlicher Paketverlust in Kauf genommen werden kann
- Keine Beeinflussung der Datenrate durch Fluss- und Staukontrollmechanismen (kann Vorteile haben, siehe Übung)

Nachteile von UDP:

- Keine Zusicherung irgendeiner Form von Dienstqualität (beliebig hohe Fehlerrate)
- Datagramme können out-of-order ausgeliefert werden (beispielsweise bei Verwendung mehrerer Pfade zu einem Ziel)
- Keine Flusskontrolle (schneller Sender kann langsamen Empfänger überfordern)
- Keine Staukontrollmechanismen (Überlast im Netz führt zu hohen Verlusten)

Einsatz von UDP

UDP wird überall dort eingesetzt, wo

- gelegentlicher Verlust von Datagrammen tolerierbar ist bzw. durch höhere Schichten wieder ausgeglichen wird oder
- ein zeitaufwendiger Verbindungsaufbau, wie er bei anderen Transportprotokollen benötigt wird, nicht tolerierbar ist.

Beispiele:

Namesauflösung mittels Domain Name System (DNS)

- Mittels DNS werden „Webadressen“ wie www.tum.de in IP-Adressen übersetzt.
- Bevor die Namensauflösung nicht abgeschlossen ist, kann auch keine Verbindung zum Ziel aufgebaut werden (→ Zeitverzögerung).

Datenverkehr mit Echtzeitanforderungen

- Mechanismen zur Fluss- und Staukontrolle können nicht-deterministische Latenzen einführen.
- Zu spät ankommende Daten haben hier häufig keine Relevanz mehr.

Google's QUIC

- Protokoll zur Beschleunigung TLS 1.3 verschlüsselter Verbindungen.
- Fehlende Mechanismen von UDP werden auf der Anwendungsschicht implementiert.
- Als Folge müssen Anwendung QUIC direkt implementieren (oder User-Space Libraries verwenden). Es gibt keine direkte Implementierung von QUIC im Kernel / Betriebssystem.

Verbindungsorientierte Übertragung

63

NETZWERKTECHNIK / SEMESTER 1 und 2

Verbindungsorientierte Übertragung

Grundlegende Idee: Linear durchnummerierte Segmente mittels **Sequenznummern** im Protokollheader

Sequenznummern ermöglichen insbesondere

- Bestätigung erfolgreich übertragener Segmente,
- Identifikation fehlender Segmente,
- erneutes Anfordern fehlender Segmente und
- Zusammensetzen der Segmente in der richtigen Reihenfolge.

Probleme: Sender und Empfänger müssen

- sich zunächst synchronisieren (Austausch der initialen Sequenznummern) und
- Zustand halten (aktuelle Sequenznummer, bereits bestätigte Segmente, ...).

Verbindungsphasen:

1. **Verbindungsaufbau (Handshake)**
2. **Datenübertragung**
3. **Verbindungsabbau (Teardown)**

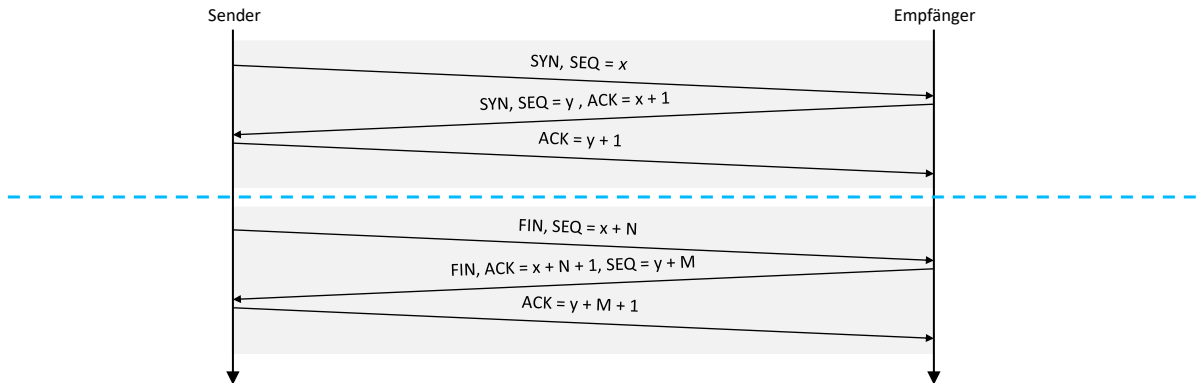
Vereinbarungen: Wir gehen zunächst davon aus,

- dass stets ganze Segmente bestätigt werden und
- dass in einer Quittung das nächste erwartete Segment angegeben wird.

64

Aufbau und Abbau einer Verbindung

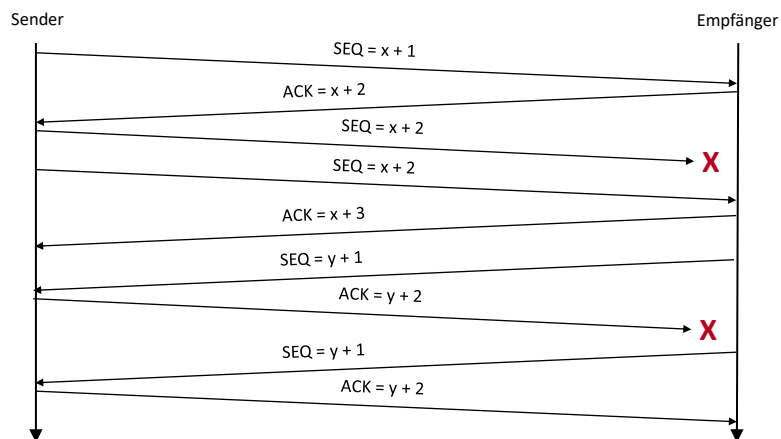
Diese Art des Verbindungsaufbaues bezeichnet man als 3-Way-Handshake.



65

Übertragungsphase

Diese Art des Verbindungsaufbaues bezeichnet man als 3-Way-Handshake.



66

Sliding-Window-Verfahren

Bisher:

- Im vorherigen Beispiel hat der Sender stets nur ein Segment gesendet und dann auf eine Bestätigung gewartet
- Dieses Verfahren ist ineffizient, da abhängig von der Umlaufverzögerung (Round Trip Time, RTT) zwischen Sender und Empfänger viel Bandbreite ungenutzt bleibt („Stop and Wait“-Verfahren)

Idee: Teile dem Sender mit, wie viele Segmente nach dem letzten bestätigten Segment auf einmal übertragen werden dürfen, ohne dass der Sender auf eine Bestätigung warten muss.

Vorteile:

- Zeit zwischen dem Absenden eines Segments und dem Eintreffen einer Bestätigung kann effizienter genutzt werden
- Durch die Aushandlung dieser **Fenstergrößen** kann der Empfänger die Datenrate steuern → **Flusskontrolle**
- Durch algorithmische Anpassung der Fenstergröße kann die Datenrate an die verfügbare Datenrate auf dem Übertragungspfad zwischen Sender und Empfänger angepasst werden → **Staukontrolle**

Sliding-Window-Verfahren

Zur Notation:

Sender und Empfänger haben denselben Sequenznummernraum $S = \{0, 1, 2, \dots, N - 1\}$.

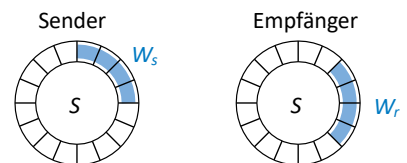
Beispiel: $N = 16$

Sendefenster (Send Window) $W_s \subset S, |W_s| = w_s$:

Es dürfen w_s Segmente nach dem letzten bestätigten Segment auf einmal gesendet werden.

Empfangsfenster (Receive Window) $W_r \subset S, |W_r| = w_r$: Sequenznummern der Segmente, die als nächstes akzeptiert werden.

Sende- und Empfangsfenster „verschieben“ und überlappen sich während des Datenaustauschs.



Sliding-Window-Verfahren

Vereinbarungen:

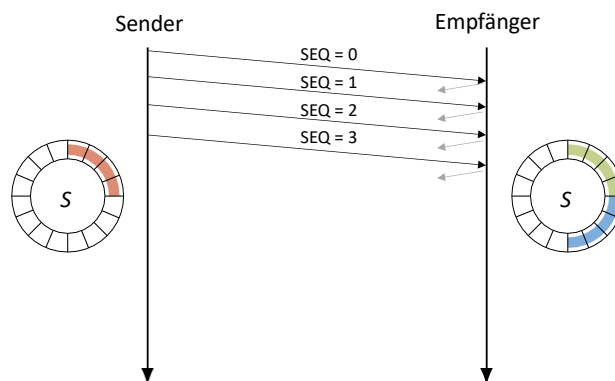
- Eine Bestätigung $ACK = m + 1$ bestätigt alle Segmente mit $SEQ \leq m$. Dies wird als **kumulative Bestätigung** bezeichnet.
- Gewöhnlich löst **jedes erfolgreich empfangene** Segment das Senden einer Bestätigung aus, wobei stets das nächste erwartete Segment bestätigt wird. Dies wird als **Forward Acknowledgement** bezeichnet.

Wichtig:

- In den folgenden Grafiken sind die meisten Bestätigungen zwecks Übersichtlichkeit nur angedeutet (graue Pfeile).
- Die Auswirkungen auf Sende- und Empfangsfenster beziehen sich nur auf den Erhalt der schwarz eingezeichneten Bestätigungen.
- Dies ist äquivalent zur Annahme, dass die angedeuteten Bestätigungen verloren gehen.

Sliding-Window-Verfahren

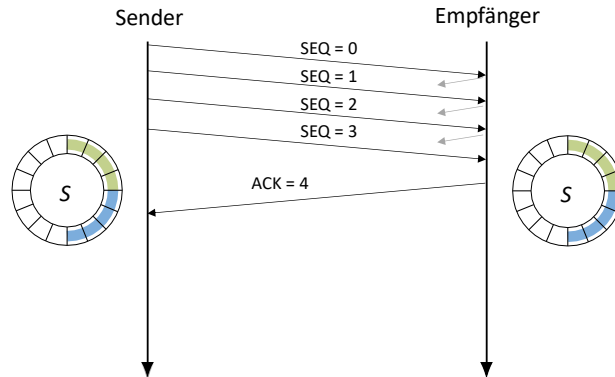
- Sendefenster W_s bzw. Empfangsfenster W_r
- gesendet aber noch nicht bestätigt
- gesendet und bestätigt/empfangen



NETZWERKTECHNIK / SEMESTER 1 und 2

Sliding-Window-Verfahren

- Sendefenster W_s bzw. Empfangsfenster W_r
- gesendet aber noch nicht bestätigt
- gesendet und bestätigt/empfangen



[Quelle: Grundlagen Rechnernetze und Verteilte Systeme (GRNVS), Prof. Dr.-Ing. Georg Carle, TUM, 2025]

tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

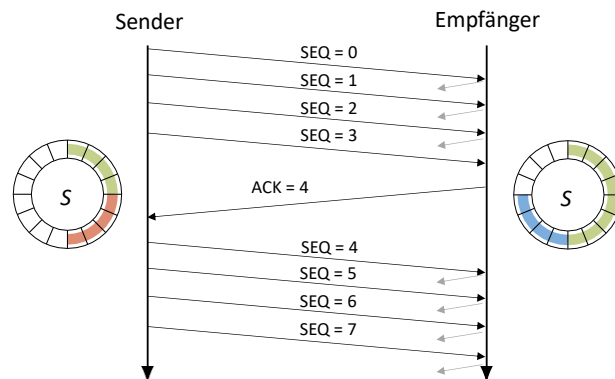
71

71

NETZWERKTECHNIK / SEMESTER 1 und 2

Sliding-Window-Verfahren

- Sendefenster W_s bzw. Empfangsfenster W_r
- gesendet aber noch nicht bestätigt
- gesendet und bestätigt/empfangen



[Quelle: Grundlagen Rechnernetze und Verteilte Systeme (GRNVS), Prof. Dr.-Ing. Georg Carle, TUM, 2025]

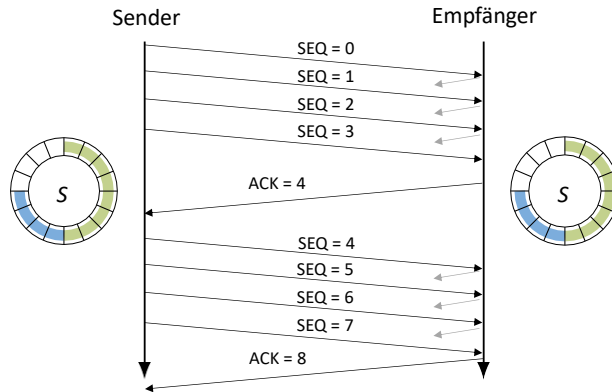
tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

72

72

Sliding-Window-Verfahren

- Sendefenster W_s bzw. Empfangsfenster W_r
- gesendet aber noch nicht bestätigt
- gesendet und bestätigt/empfangen



Sliding-Window-Verfahren

Wie wird mit Segmentverlusten umgegangen?

Zwei Möglichkeiten:

Go-Back-N

- Akzeptiere stets nur die nächste erwartete Sequenznummer
- Alle anderen Segmente werden verworfen

Selective-Repeat

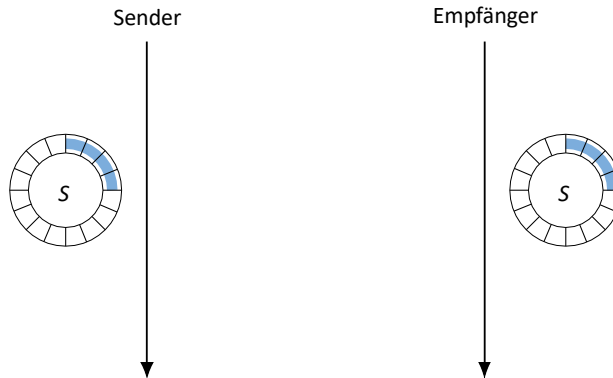
- Akzeptiere alle Sequenznummern, die in das aktuelle Empfangsfenster fallen
- Diese müssen gepuffert werden, bis fehlende Segmente erneut übertragen wurden

Wichtig:

- In beiden Fällen muss der Sequenznummernraum so gewählt werden, dass wiederholte Segmente eindeutig von neuen Segmenten unterschieden werden können.

NETZWERKTECHNIK / SEMESTER 1 und 2

Sliding-Window-Verfahren

Go-Back-N: $N = 16$, $w_s = 4$, $w_r = 4$ 

[Quelle: Grundlagen Rechnernetze und Verteilte Systeme (GRNVS), Prof. Dr.-Ing. Georg Carle, TUM, 2025]

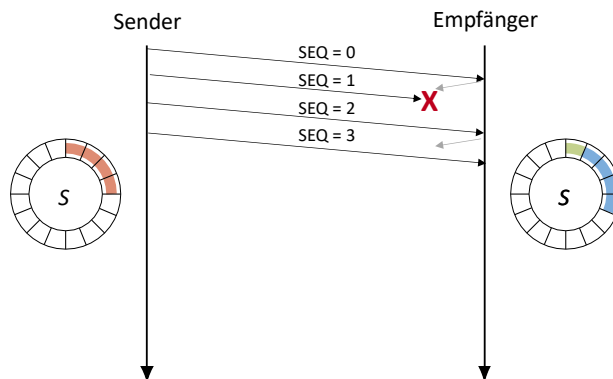
tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

75

75

NETZWERKTECHNIK / SEMESTER 1 und 2

Sliding-Window-Verfahren

Go-Back-N: $N = 16$, $w_s = 4$, $w_r = 4$ 

[Quelle: Grundlagen Rechnernetze und Verteilte Systeme (GRNVS), Prof. Dr.-Ing. Georg Carle, TUM, 2025]

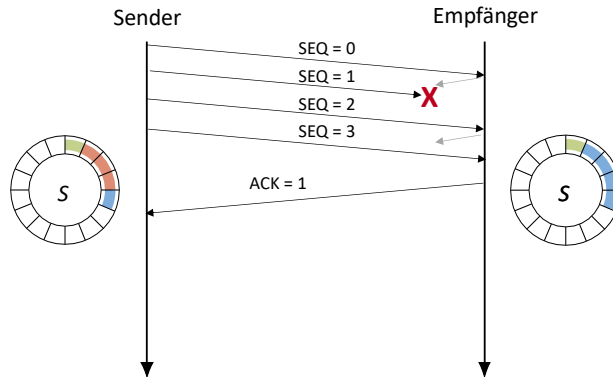
tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

76

76

NETZWERKTECHNIK / SEMESTER 1 und 2

Sliding-Window-Verfahren

Go-Back-N: $N = 16$, $w_s = 4$, $w_r = 4$ 

[Quelle: Grundlagen Rechnernetze und Verteilte Systeme (GRNVS), Prof. Dr.-Ing. Georg Carle, TUM, 2025]

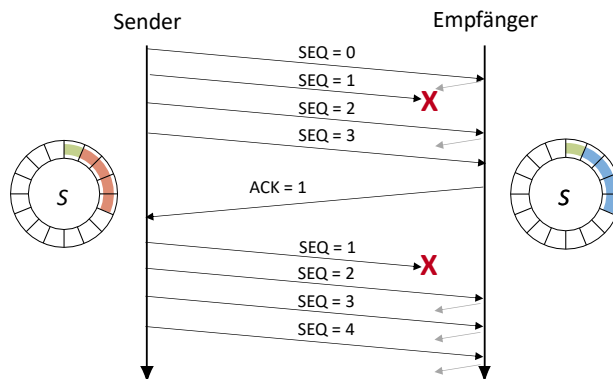
tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

77

77

NETZWERKTECHNIK / SEMESTER 1 und 2

Sliding-Window-Verfahren

Go-Back-N: $N = 16$, $w_s = 4$, $w_r = 4$ 

[Quelle: Grundlagen Rechnernetze und Verteilte Systeme (GRNVS), Prof. Dr.-Ing. Georg Carle, TUM, 2025]

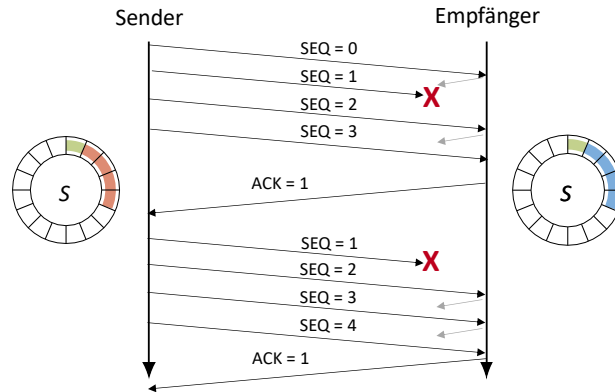
tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

78

78

NETZWERKTECHNIK / SEMESTER 1 und 2

Sliding-Window-Verfahren

Go-Back-N: $N = 16$, $w_s = 4$, $w_r = 4$ 

[Quelle: Grundlagen Rechnernetze und Verteilte Systeme (GRNVS), Prof. Dr.-Ing. Georg Carle, TUM, 2025]

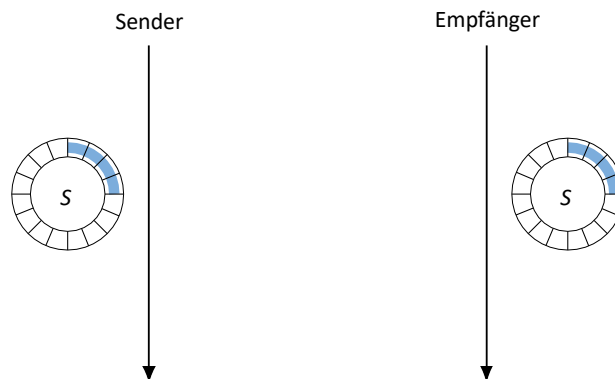
tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

79

79

NETZWERKTECHNIK / SEMESTER 1 und 2

Sliding-Window-Verfahren

Selective Repeat: $N = 16$, $w_s = 4$, $w_r = 4$ 

[Quelle: Grundlagen Rechnernetze und Verteilte Systeme (GRNVS), Prof. Dr.-Ing. Georg Carle, TUM, 2025]

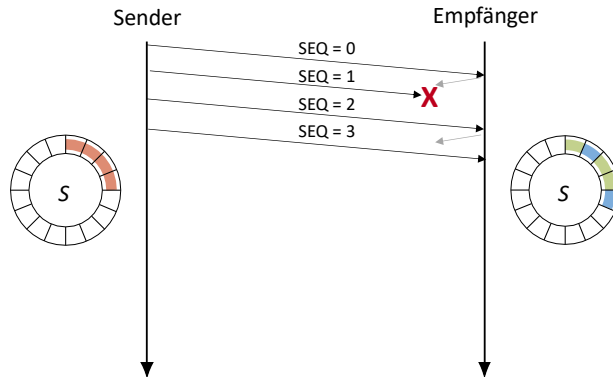
tgm | Technologisches Gewerbemuseum | Höhere technische Bundes-Lehr- und Versuchsanstalt

80

80

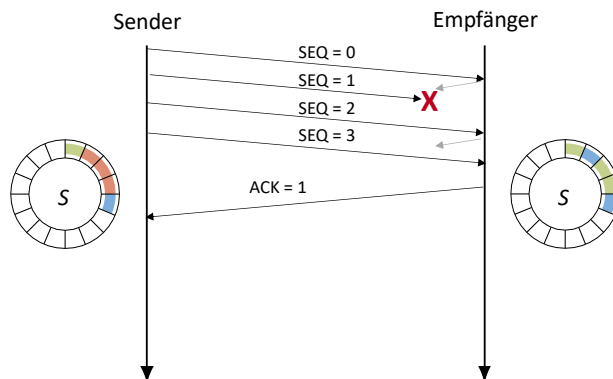
Sliding-Window-Verfahren

Selective Repeat: $N = 16$, $w_s = 4$, $w_r = 4$



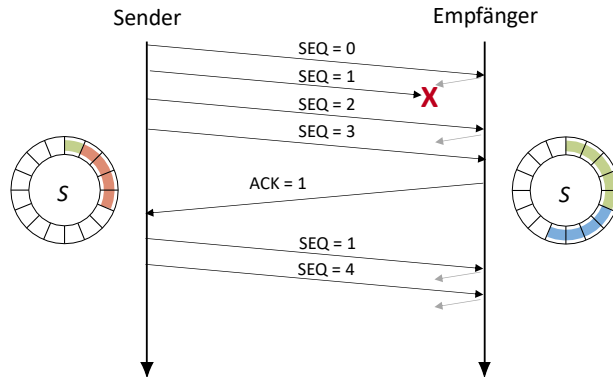
Sliding-Window-Verfahren

Selective Repeat: $N = 16$, $w_s = 4$, $w_r = 4$



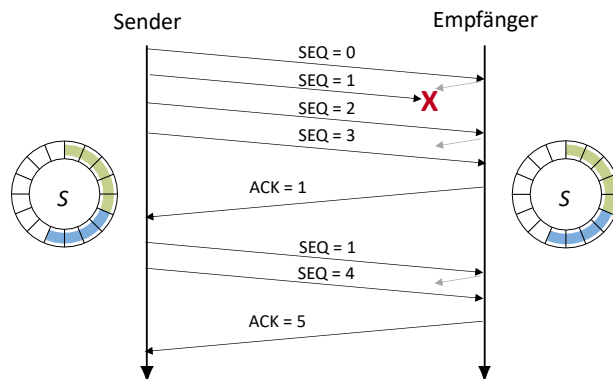
Sliding-Window-Verfahren

Selective Repeat: $N = 16$, $w_s = 4$, $w_r = 4$



Sliding-Window-Verfahren

Selective Repeat: $N = 16$, $w_s = 4$, $w_r = 4$



TCP

85

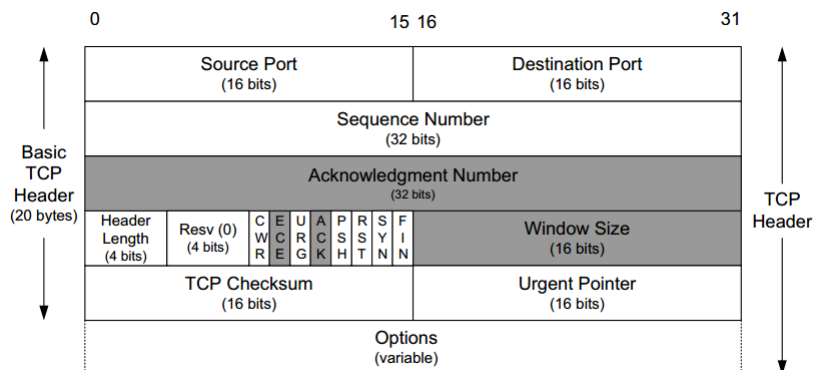
NETZWERKTECHNIK / SEMESTER 1 und 2

Transmission Control Protocol (TCP)

TCP ist das dominierende Transportprotokoll im Internet (rund 90 % des Datenverkehrs im Internet).

Es bietet

- gesicherte / stromorientierte Übertragung mittels **Sliding-Window** und **Selective Repeat** sowie
- Mechanismen zur Fluss- und Staukontrolle.



86

Transmission Control Protocol (TCP)

- **Quell- und Zielpport** werden analog zu UDP verwendet.
- **Sequenz- und Bestätigungsnummer** dienen der gesicherten Übertragung. Es werden bei TCP nicht ganze Segmente sondern einzelne Bytes bestätigt (stromorientierte Übertragung).
- **(Data) Offset** gibt die Länge des TCP-Headers in Vielfachen von 4 B an. Der TCP-Header hat variable Länge (Optionen, vgl. IPv4-Header).
- **Reserved** hat in bisherigen TCP-Versionen keine Verwendung. Muss auf 0 gesetzt werden, so dass zukünftige TCP-Versionen bei Bedarf das Feld nutzen können.
- **Flag URG („urgent“)** (selten verwendet). Ist das Flag gesetzt, werden die Daten im aktuellen TCP-Segment beginnend mit dem ersten Byte bis zu der Stelle, an die das Feld Urgent Pointer zeigt, sofort an höhere Schichten weitergeleitet.
- **Flag ACK („acknowledgement“)** Ist das Flag gesetzt, handelt es sich um eine Empfangsbestätigung.
 - Bestätigungen können bei TCP auch „huckepack“ (engl. **piggy backing**) übertragen werden, d. h. es werden gleichzeitig Nutzdaten von A nach B übertragen und ein zuvor von B nach A gesendetes Segment bestätigt.
 - Die Acknowledgement-Number gibt bei TCP stets **das nächste erwartete Byte** an.

Transmission Control Protocol (TCP)

- **Flag PSH („push“)**
 - Ist das Flag gesetzt, werden sende- und empfangsseitige Puffer des TCP-Stacks umgangen.
 - Sinnvoll für interaktive Anwendungen (z. B. Telnet-Verbindungen).
- **Flag RST („reset“)**
 - Dient dem Abbruch einer TCP-Verbindung ohne ordnungsgemäßen Verbindungsabbau.
- **Flag SYN („synchronization“)**
 - Ist das Flag gesetzt, handelt es sich um ein Segment, welches zum Verbindungsaufbau gehört (initialer Austausch von Sequenznummern).
 - Ein gesetztes SYN-Flag inkrementiert Sequenz- und Bestätigungsnummern um 1 obwohl keine Nutzdaten transportiert werden.
- **Flag FIN („finish“)**
 - Ist das Flag gesetzt, handelt es sich um ein Segment, welches zum Verbindungsabbau gehört.
 - Ein gesetztes FIN-Flag inkrementiert Sequenz- und Bestätigungsnummern um 1 obwohl keine Nutzdaten transportiert werden.

Transmission Control Protocol (TCP)

- **Receive Window**
 - Größe des aktuellen Empfangsfensters W_r in Byte.
 - Ermöglicht es dem Empfänger, die Datenrate des Senders zu drosseln.
- **Checksum**
 - Prüfsumme über Header und Daten.
 - Wie bei UDP wird zur Berechnung ein Pseudo-Header verwendet.
- **Urgent Pointer (selten verwendet)**
 - Gibt das Ende der „Urgent-Daten“ an, welche unmittelbar nach dem Header beginnen und bei gesetztem URG-Flag sofort an höhere Schichten weitergereicht werden sollen.
- **Options**
 - Zusätzliche Optionen, z. B. Window Scaling, selektive Bestätigungen oder Angabe der Maximum Segment Size (MSS).

Anmerkungen zur Maximum Segment Size (MSS)

- Die MSS gibt die maximale Größe eines TCP-Segments (Nutzdaten ohne TCP-Header) an.
- Zum Vergleich gibt die MTU (Maximum Transfer Unit) die maximale Größe der Nutzdaten aus Sicht von Schicht 2 an (alles einschließlich des IP-Headers).
- In der Praxis sollte die MSS so gewählt werden, dass keine IP-Fragmentierung beim Senden notwendig ist¹

Beispiele:

MSS bei FastEthernet

- MTU beträgt 1500 B.
- Davon entfallen 20 B auf den IPv4-Header und weitere 20 B auf den TCP-Header (sofern keine Optionen verwendet werden).
- Die sinnvolle MSS beträgt demnach 1460 B.

DSL-Verbindungen

- Zwischen Ethernet- und IP-Header wird ein 8B langer PPPoE-Header eingefügt.
- Demzufolge sollte die MSS auf 1452B reduziert werden.

VPN-Verbindungen

- Abhängig vom eingesetzten Verschlüsselungsverfahren sind weitere Header notwendig.
- Die sinnvolle MSS ist hier nicht immer offensichtlich.

Fluss und Staukontrolle bei TCP

91

NETZWERKTECHNIK / SEMESTER 1 und 2

Fluss- und Staukontrolle bei TCP

TCP-Flusskontrolle

Ziel der Flusskontrolle ist es, Überlastsituationen beim Empfänger zu vermeiden. Dies wird erreicht, indem der Empfänger eine Maximalgröße für das Sendefenster des Senders vorgibt.

- Empfänger teilt dem Sender über das Feld Receive Window im TCP-Header die aktuelle Größe des Empfangsfensters W_r mit.
- Der Sender interpretiert diesen Wert als die maximale Anzahl an Byte, die ohne Abwarten einer Bestätigung übertragen werden dürfen.
- Durch Herabsetzen des Wertes kann die Übertragungsrate des Senders gedrosselt werden, z. B. wenn sich der Empfangspuffer des Empfängers füllt.

TCP-Staukontrolle

Ziel der Staukontrolle ist es, Überlastsituationen im Netz zu vermeiden. Dazu muss der Sender Engpässe im Netz erkennen und die Größe des Sendefensters entsprechend anpassen.

- Zu diesem Zweck wird beim Sender zusätzlich ein **Staukontrollfenster** (engl. Congestion Window) W_c eingeführt, dessen Größe wir mit W_c bezeichnen:
- W_c wird vergrößert, solange Daten verlustfrei übertragen werden.
- W_c wird verkleinert, wenn Verluste auftreten.
- Für das tatsächliche Sendefenster gilt stets $w_s = \min\{w_c, w_r\}$.

92

TCP-Staukontrolle

Man unterscheidet bei TCP grundsätzlich zwischen zwei Phasen der Staukontrolle:

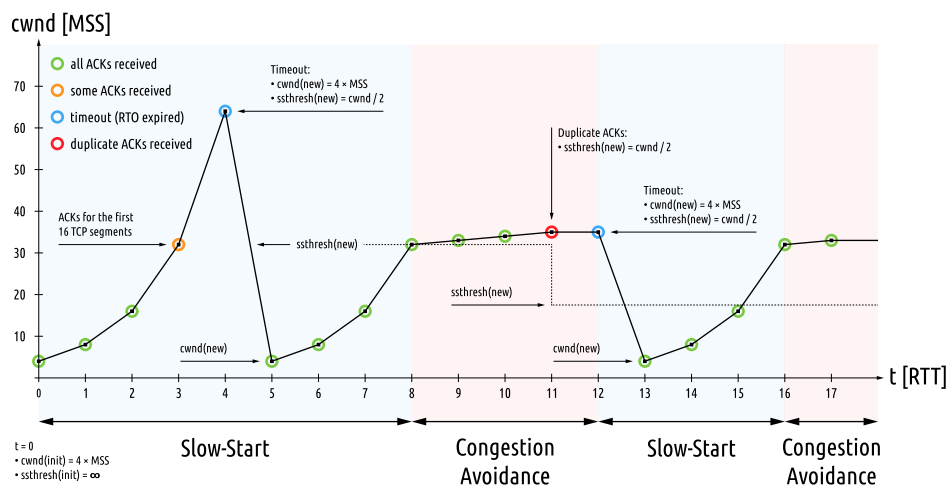
Slow-Start:

- Für jedes bestätigte Segment wird W_c um eine MSS vergrößert.
- Dies führt zu **exponentiellem Wachstum** des Staukontrollfensters bis ein Schwellwert (engl. **Congestion Threshold**) erreicht ist.
- Danach wird mit der Congestion-Avoidance-Phase fortgefahren.

Congestion Avoidance:

- Für jedes bestätigte Segment wird W_c lediglich um $(1/w_c)$ MSS vergrößert, d. h. nach Bestätigung eines vollständigen Staukontrollfensters um genau eine MSS.
- Ein vollständiges Fenster kann frühestens nach 1 RTT bestätigt sein.
- Dies führt zu **linearem Wachstum** des Staukontrollfensters in der RTT.

TCP-Staukontrolle



TCP-Staukontrolle

TCP ermöglicht gesicherte Verbindungen. Die Protokollmechanismen zur Fehlerkontrolle wurden im Hinblick auf Überlast im Netz entwickelt:

- TCP interpretiert den Verlust von Segmenten (Daten und Bestätigungen) stets als eine Folge einer Überlastsituation im Netzwerk (und nicht als Folge von Bitfehlern einer unzuverlässigen Übertragung).
- In der Folge reduziert TCP die Datenrate.
- Handelt es sich bei den Paketverlusten jedoch um die Folge von Bitfehlern, so wird die Datenrate unnötiger Weise gedrosselt.
- Durch die ständige Halbierung der Datenrate bzw. neue Slow-Starts kann das Sendefenster nicht mehr auf sinnvolle Größen anwachsen.
- In der Praxis ist TCP bereits mit 1 % Paketverlust, der nicht auf Überlast zurückzuführen ist, überfordert.

Die Schichten 1 – 3 müssen eine für TCP „ausreichend geringe“ Paketfehlerrate bereitstellen.

- In der Praxis bedeutet dies, dass Verlustwahrscheinlichkeiten in der Größenordnung von 10^{-3} und niedriger notwendig bzw. anzustreben sind.
- Bei Bedarf müssen zusätzliche Bestätigungsverfahren auf Schicht 2 zum Einsatz kommen, z. B. IEEE 802.11.